A Study Report
on
Monarch: Google’s Planet-Scale In-Memory Time Series Database

By Datta Ksheeraj

21CS30037

Introduction

Google operates one of the largest and most complex infrastructures in the world, with millions of servers across data centers worldwide. Monitoring and managing performance metrics across this massive infrastructure requires a database capable of handling petabytes of data and millions of queries per second.

Google uses various types of databases, each suited for specific tasks. Examples include Spanner, Bigtable, Colossus, BlobStore etc. There should be monitoring system over all these databases. Generally a TSDB (Time-Series Database is used which captures events with time stamps

Monarch is such a database which keeps an eye on all existing database systems and is used as an alerting system too to report for any inconsistencies seen.

The Problem

Google’s infrastructure comprised numerous clusters, each hosting thousands of servers running various applications. The complexity and scale of these clusters made it challenging to monitor system health effectively. Existing monitoring solutions lacked the ability to provide real-time insights into the status of individual servers, services, and applications. This often resulted in:

- Need to get a time-based collection of data.
- Difficulty in identifying failing or underperforming nodes.
- Slow response times to incidents due to limited visibility.
- Overwhelming volumes of alerts, many of which were not actionable.

Borgmon was designed as a time-series database to provide comprehensive cluster monitoring by implementing: Real Time Monitoring, Centralized Data Collection, Intelligent Alerting.

But Later on what happened is Google's infrastructure has grown rapidly that no more the centralised architecture worked, To make Borgmon work in such situations engineers had to work harder to make things reliable. Also Borgmon wasnt provided with fault tolerance and was lacking query optimisaations which just made the system slow in terms of performance and more vulnerable if at all crashes happen in some part of system

So by that time the following problems seemed unsolved with Borgmon:

- **Scalability and Performance**
- **Availability and Fault Tolerance**
- **All kinds of optimisations on query** (This isnt a problem in itself with Borgmon but just to put it up)
- **Storage Issue** (There werent any methods of compressing large data (which has grown to rate of petabytes per unit time)
- **Load Balancing** (This isnt implemented as such in Borgmon but is indeed a performance booster)

Core Solution Ideas

For all the problems mentioned previosly, We now know the solutions which I would like to list, The main problem was to develop such sytem which integrates all the known solutions . Monarch was such a result of all such decisions made to overcome those problems

Now listing the existing solutions we know for the problems:

Scalability and Performance: The system instead of having a centralized architecture, now should have a decentralized one. This now adds on complexity to development of such architecture hence is a good trade off (Definitely if putting more effort brings in a system which is more scalable Why not?)

Fault Tolerance: Simple solution for this is to maintain replicas which is indeed done in Monarch. Also to improve performance we maintain each replica to a different level of granularity (detailing).

Storage Issue: With petabytes of data needed to be stored we need to introduce some sort of compression or new data structures to hold data effectively and in cheapest possible manner. For some encoding techniques were used which serve the purpose.

Load Balancing: Again this is a very important performance optimizing technique which can bring down performance by large scale if not implemented properly. In Monarch this is done as part of ingestion of data and can also be done individually by each local storage unit as we shall see later.

Having seen all the basic solutions we have in our mind , We will see that Monarch was indeed implemented keeping in mind all these solutions

Implementation

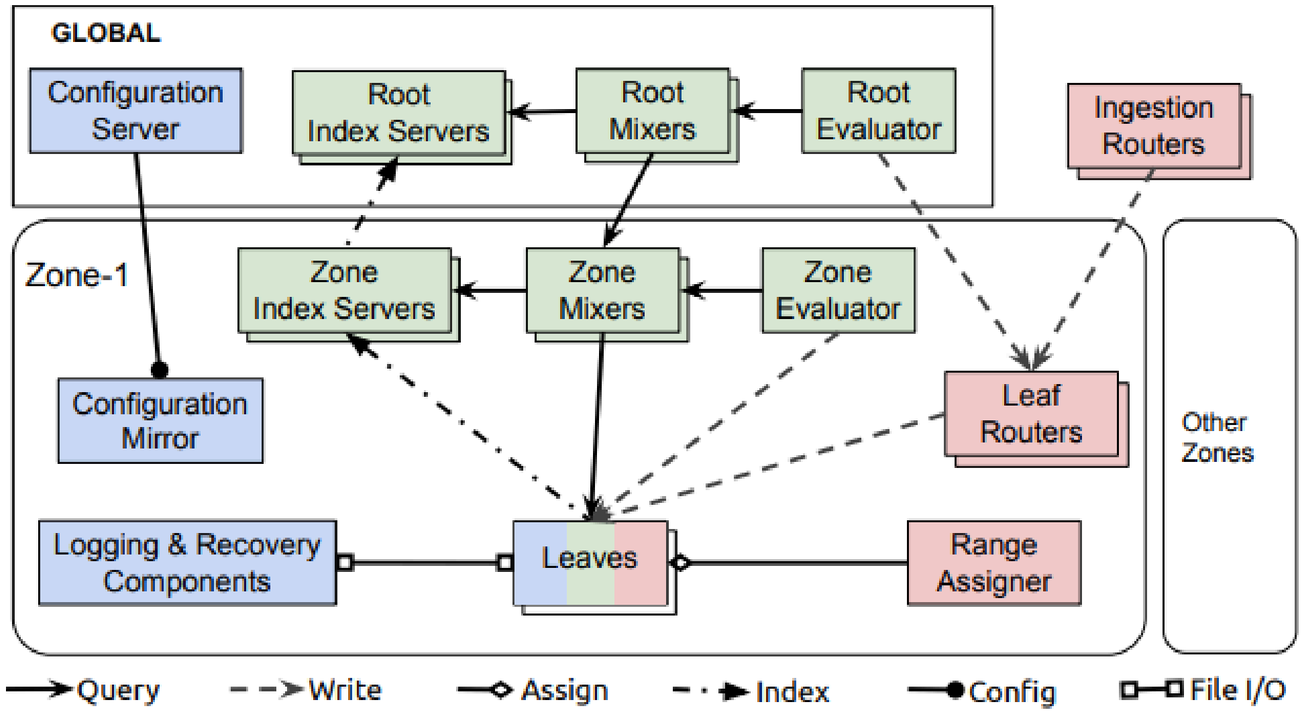
Before we get into the part where we will see how Monarch managed to solve all those problems, We would first look into the implementation details so that things become more clear. Having said that, the implementation will be put in here with minimal description as I have covered all of that in the report in detail. But nevertheless all the components will be covered.

The basic outline of this part includes:

- Description of Components of Monarch
- How is Data stored in Monarch

Then we will be going to see how query execution takes place in Monarch

Components: Monarch's architecture includes components categorized by their primary functions: those responsible for storing state, those dedicated to data ingestion, and those focused on query execution.



State-Holding Components:

- Leaves**:** primary storage for monitoring data
- Recovery Logs: store monitoring data on disk, mirroring the data held in leaves
- Global configuration server and zonal mirrors manage configuration data

Data Ingestion Components

- Ingestion routers direct data to leaf routers within the correct Monarch zone, relying on information embedded in time series keys for accurate routing.
- Leaf routers receive data bound for a specific zone and forward it to the leaves for storage.
- Range assigners handle the distribution of data across leaves, balancing the load within each zone.

Query Execution Components

- Mixers divide queries into smaller sub-queries routed to and processed by leaves, then combine the results. Queries can be initiated at the root level (handled by root mixers) or at the zone level (handled by zone mixers). Rootlevel queries require both root and zone mixers.
- Index servers provide indexing for each zone and leaf, guiding the distributed execution of queries.
- Evaluators periodically execute standing queries (see Section 5.2) by dispatching them to mixers, then write the results back to leaves.

Data Storage:

Monarch organizes monitoring data into tables that store time series, which are sequences of values recorded over time. Each table includes two main types of columns:

- Key columns: These define unique identifiers for each time series, such as the specific machine or service being tracked.
- Value column: This stores the historical data points (e.g., memory usage or latency) recorded over time for each time series.

Key columns, also referred to as fields, have two sources: targets and metrics.

The data is stored in two formats:

- Leaves are the components where the actual monitoring data is stored in memory
- Logs are persistent stores that can be used to replay the events in case of component failures

How data travels from client into the system:

Client to Ingestion Router

- A client application sends time series data to a nearby ingestion router. Monarch's ingestion routers are spread across clusters, allowing data to be gathered close to its origin, thus reducing latency.
- Clients frequently utilize Monarch's instrumentation library, which regulates the data write frequency to comply with Monarch's retention policies.

Ingestion Router to Leaf Router

- Each ingestion router extracts the location field of the target to determine the appropriate zone for the data, organizing it by geographic or logical regions.
- After identifying the target zone, the ingestion router forwards the data to a leaf router within that zone. Zone mappings are dynamically updated.

Leaf Router to Leaf Node

- Within the assigned zone, the leaf router directs data to specific leaf nodes. These leaf nodes manage target ranges, with data sharded lexicographically according to target strings.
- Each leaf router keeps an updated range map that indicates which leaf node is responsible for each target range. This map is periodically refreshed by updates from leaf nodes, ensuring uninterrupted data collection even if the range assigner experiences temporary issues.

Before going to the part where we will see how the query execution takes place, we will see some implementation details which actually solved those problems which were stated earlier

Replication:

- Replication Policy: Monarch allows users to select the number of replicas per target (ranging from 1 to 3), enabling a trade-off between availability and storage cost.
- Granularity Control: Users can choose to retain different levels of data detail for each replica, such as retaining only a fine-grained view on one replica and a coarser view on others.

Load Balancing:

- The range assigner actively balances the load by moving, splitting, or merging ranges as needed based on CPU load and memory usage across leaf nodes

Data Availability:

- During the transfer process, both the source and destination leaves temporarily collect and log data for range R to avoid any data loss and ensure continuous availability.(Interesting optimisation)

Direct Updates from Leaves: Leaves keep leaf routers informed about range assignments, rather than relying on the range assigner for updates(Interesting optimisation) which makes system run smoothly despite failure of range assigner

Scalability and Performance:

- Monarch's architecture, as seen in implementation is a decentralized global system that enables independent control over various parts, making it convenient for engineers to operate and debug.
- For high scalability, Monarch uses a field hints index (FHI), stored in index servers, to limit the load when sending a query from parent to children by skipping irrelevant children (those without input data to the particular query). This is implemented using n-grams for the data and storing what all leaves contain that.

Storage: For the vast incoming data which is at rate of petabytes per unit time, Monarch used delta-time encoding

Monarch prioritizes high availability and partition tolerance over consistency. Interactions with strongly consistent databases, such as Spanner, can result in lengthy blocks, which are not suitable for Monarch. This blocking could prolong the meantimeto-detection and mean-time-to-mitigation during potential outages. To ensure timely alert delivery, Monarch focuses on providing the most recent data quickly

Query Execution

In Monarch, there are two types of queries:

- Ad hoc Queries These are one-time queries from users outside the system.
- Standing Queries These are periodic queries whose results are stored back in Monarch. Standing queries are used to generate alerts or pre-process data for faster and more cost-effective access later. They can be processed at either the regional zone level or the global root level, based on the type of data and query requirements. Most standing queries are handled at the zone level, making them more efficient and resilient to network issues.

Queries are processed in a three-level hierarchy:

- Root Mixer: Receives the query and distributes it to zone mixers.
- Zone Mixers: Send the query to relevant leaf nodes based on an index.
- Leaf Nodes: Process the data closest to the source.

Query Pushdown: Monarch minimizes data transfer by processing as much of the query as possible at the lowest level. This is called “query pushdown” and improves speed by:

- Reducing the amount of data transferred to higher levels.
- Allowing more concurrent processing.

For example, if a query only involves data from one zone, it can be fully processed there without needing root-level involvement. This pushdown approach makes 95% of standing queries complete within the zone, which also prevents cross-region data traffic and reduces latency.

Result Aggregation:

- Leaf Nodes process data closest to the source. They handle their assigned ”target range” and perform basic filtering and aggregation to reduce data before sending it up.
- Zone Mixers gather results from multiple leaf nodes in their region. They combine and further aggregate data, then send only summarized information to the root mixer.
- Root Mixer collects data from zone mixers. It completes any remaining aggregation, checks security, and applies final optimizations.

My Observations

While Monarch prioritizes high availability and partition tolerance, the trade-off with data consistency could lead to stale reads. This seemed more bold decision to me, instead we should be using better consistency model which does different things based on situation.

For this I thought of implementing a Reinforcement Learning Model which slowly learns the data it handles and performs accordingly.

Also user was given choice to choose the number of replicas and level of granularity. But when scalability is considered and lot of users operate at same time they might over estimate the usage which unnecessarily creates extra load if all created maximum number of replicas.

For this the control given to users must be looked into again, there must be control mechanisms asking users about the data they are going to habdle and based on that the system should have some percentage of controlling. Even complete control to the system itself is a waste